Structured Computer Architecture
Chapter 2.11: Processors
Third-Order Systems (3-OS)

An Order-Based Approach

John Glossner and Gheorghe M. Ștefan

Based on the textbook

September 14, 2025

Outline

Learning Objectives

By the end of this chapter, you will be able to:

- Understand third-order systems (3-OS) and the third feedback loop
- Analyze the structure of elementary processors
- Design RALU (Registers with ALU) components
- Understand CROM (Control ROM) operation
- Explain micro-programming concepts
- Recognize context-free language processing capabilities

Evolution to 3-OS: The Third Loop

- **Previous Systems Review**:
  - **0-OS**: No loops, combinational circuits
  - **1-OS**: One loop, memory capability
  - **2-OS**: Two loops, autonomous behavior

- **3-OS Innovation**: Third feedback loop
  - Can be closed through combinational circuit
  - Can be closed through memory circuit
  - **Focus**: Closed through automaton over automaton
  - Enables processor functionality

- **Key Achievement**:
  - Complex computational behavior
  - Context-free language recognition
  - Elementary processor implementation

Context-Free Languages and Processors

- **Language Hierarchy**:
  - **Type 3 (Regular)**: Recognized by finite automata
  - **Type 2 (Context-Free)**: Require more powerful systems
  - **Type 1 (Context-Sensitive)**: Even more complex
  - **Type 0 (Unrestricted)**: Turing machine equivalent

- **3-OS Capability**:
  - Recognizes context-free languages
  - Processes nested structures
  - Handles recursive patterns
  - Enables programming language parsing

- **Practical Applications**:
  - Compiler design
  - Expression evaluation
  - Nested procedure calls
  - Balanced parentheses checking

Beyond Finite Automata Limitations

- **Finite Automaton Limitation**:
  - Cannot recognize $\{1^n 0^n | n > 0\}$
  - Fixed number of states independent of n
  - No counting capability

- **Counter-Expanded Automaton (CEA)**:
  - Finite automaton + reversible counter
  - Counter provides zero flag to automaton
  - Creates 3-OS system
  - Enables context-free language recognition

- **Enhanced Capabilities**:
  - Counting and comparison operations
  - Stack-like behavior simulation
  - Nested structure processing
  - Context-free grammar parsing

CEA Structure and Operation

Components:

- **Finite Automaton**: Control logic
- **Reversible Counter**: Counting mechanism
- **Zero Flag**: Counter state indicator
- **Command Interface**: Counter control

Counter Operations:

- **NOP**: No operation
- **UP**: Increment counter
- **DOWN**: Decrement counter
- **Zero Flag**: Indicates counter $= 0$

System Integration:

- Both FA and Counter are 2-OS
- Loop connection creates 3-OS
- Enhanced computational power

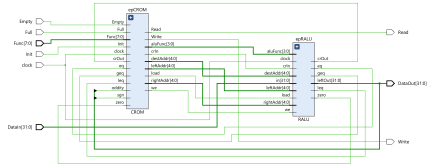
Elementary Processor Overview

Definition: Simplest version of a processor

- Implements 3-OS principles
- Demonstrates processor fundamentals
- Educational and practical value

Main Components:

- **RALU**: Registers with Arithmetic & Logic Unit
- **CROM**: Control ROM (micro-programmed control)
- **Interface**: Data input/output and control signals



Design Philosophy:

- Simple functional automaton (RALU)
- Complex control automaton (CROM)
- Clear separation of concerns

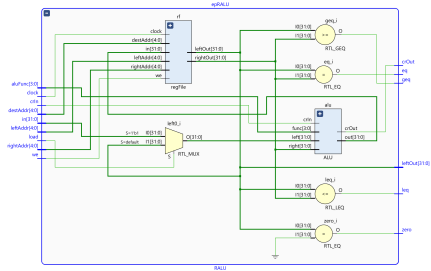
RALU: Registers with ALU

Structure:

- **Register File**: Multi-port storage
- **ALU**: Arithmetic and logic operations
- **Flag Generation**: Status indicators
- **Data Paths**: Interconnection network

Flag Outputs:

- **crOut**: Carry output from ALU
- **eq**: Equal (leftOut == rightOut)
- **leq**: Less than or equal
- **geq**: Greater than or equal
- **zero**: Equal to zero
- **sgn**: Sign bit (negative)
- **oddity**: Odd number indicator



Characteristics:

- Simple functional automaton
- Regular, predictable structure
- Low complexity relative to size
- Efficient implementation

CROM: Control ROM

Function:

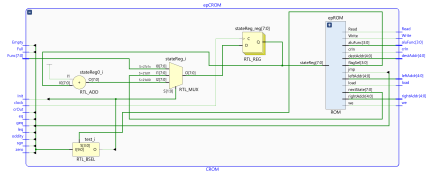
- Micro-programmed control unit
- Stores control sequences
- Implements instruction execution
- Provides system coordination

Components:

- **ROM**: Micro-instruction storage
- **Program Counter**: Address generation
- **Increment Logic**: Sequential addressing
- **Multiplexers**: Address selection

External Flags:

- **Empty**: Data source empty
- **Full**: Data receiver full



Characteristics:

- Complex control automaton
- High complexity relative to size
- Flexible programming capability
- Sophisticated behavior

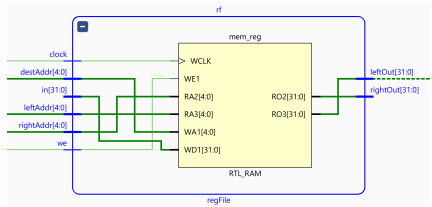
Register File Architecture

Multi-Port Design:

- Two read ports (left and right)
- One write port
- Simultaneous access capability
- Independent addressing

Interface Signals:

- **leftAddr**: Left read address
- **rightAddr**: Right read address
- **destAddr**: Write destination
- **writeEnable**: Write control
- **leftOut/rightOut**: Read outputs
- **in**: Write data input



Operation:

- Synchronous write operation
- Asynchronous read operation
- Dual-port read capability
- Register-based storage

Register File Implementation Details

- **Storage Structure**:
  - Array of registers (flip-flops)
  - Addressable storage locations
  - Parallel access capability
  - Fast read/write operations

- **Access Mechanism**:
  - Address decoders for selection
  - Multiplexers for output routing
  - Write enable gating
  - Simultaneous multi-port access

- **Design Considerations**:
  - Area vs. performance trade-offs
  - Number of registers vs. access time
  - Power consumption optimization
  - Technology mapping considerations

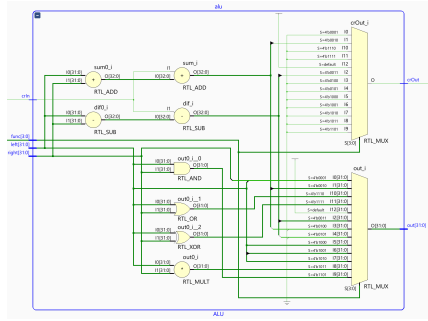
ALU Functionality

Supported Operations:

- **BIGVAL**: Concatenation operation
- **ADD**: Addition
- **SUB**: Subtraction
- **ADDCR**: Addition with carry
- **SUBCR**: Subtraction with carry
- **LSH**: Logical shift
- **ASH**: Arithmetic shift

Control Interface:

- **aluFunc[3:0]**: Operation selection
- **crIn**: Carry input
- **crOut**: Carry output flag



Design Features:

- Multi-function capability
- Flag generation
- Carry chain support
- Efficient implementation

ALU Operation Examples

- **Arithmetic Operations**:
  - **ADD**: $D(X) \leftarrow L(Y) + R(Z)$
  - **SUB**: $D(X) \leftarrow L(Y) - R(Z)$
  - **ADDCR**: Addition with carry propagation
  - **SUBCR**: Subtraction with borrow

- **Logical Operations**:
  - **BIGVAL**: $D(X) \leftarrow \{L(Y)[15:0], R(Z)[15:0]\}$
  - Bit manipulation operations
  - Boolean logic functions

- **Shift Operations**:
  - **LSH**: $D(X) \leftarrow \{1'b0, L(Y)[31:1]\}$
  - **ASH**: $D(X) \leftarrow \{L(Y)[31], L(Y)[31:1]\}$
  - Multiplication/division by powers of 2